

Day 4 Lab Sheet: A Real Database, Git and GitHub

Goal: By the end of this lab your Notes Board keeps its notes in a SQLite database, so they stay after a restart. Your project is saved with Git and is on GitHub, and your private `.env` file is **not** on GitHub.

Time: about 100 minutes, plus the quiz.

What you need: Your `notes-board` folder from Day 3, your `.env` file, and your GitHub account and password (from your homework).

If your Day 3 files are missing or broken, copy the **Day 4 starter** folder from your instructor. It has everything finished up to Day 3. The **done** folder is the finished result for today.

Part 0: Check your setup (5 minutes)

1. Open VS Code and open your `Workshop` folder.
2. Open a terminal (**Terminal**, then **New Terminal**) and choose **Command Prompt**.
3. Go into your project folder and check the tools:

```
cd notes-board
node -v
git --version
dir
```

Check:

- `node -v` shows a version that starts with `v24`.
- `git --version` shows a version number.
- `dir` shows `server.js`, `data.js`, `package.json`, `.gitignore` and a `.env` file. You can also look in the VS Code file list on the left.

Now start the Day 3 server once to prove that the starter works:

```
node server.js
```

Open `http://localhost:3000` in the browser. You see the notes from Day 3. Press **Ctrl + C** in the terminal to stop the server.

If you see `Cannot find module` : you are in the wrong folder. Use `cd` until `dir` shows `server.js` . See the Stuck Sheet.

If your `.env` file is missing: copy `.env.example` to `.env` and write a password after `NOTES_PASSWORD=` .

Part 1: A first look at SQLite (15 minutes)

SQLite is built into Node.js 24, so there is nothing to install. We do **not** have a SQLite command window, so we run SQL from a very small JavaScript file. This file is only for practice. We delete it at the end of this part.

1. In your `notes-board` folder, create a new file named `try.js` .
2. Type this code **exactly**:

```
const { DatabaseSync } = require("node:sqlite");

const db = new DatabaseSync("try.sqlite");

db.exec("CREATE TABLE IF NOT EXISTS fruits (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL)");

const insert = db.prepare("INSERT INTO fruits (name) VALUES (?)");
insert.run("Mango");
insert.run("Banana");

const rows = db.prepare("SELECT id, name FROM fruits ORDER BY name").all();
console.log(rows);

const count = db.prepare("SELECT COUNT(*) AS total FROM fruits").get();
console.log("Rows in the table:", count.total);
```

3. Save the file (**Ctrl + S**). In the terminal, run:

```
node try.js
```

Check: You see two fruits in a list, `Banana` first (because `ORDER BY name` sorts by name), and the line `Rows in the table: 2` . The text `[object: null prototype]` is only how Node.js prints a row. It is normal.

What each line did: `CREATE TABLE` made a table named `fruits`. `INSERT` added rows. `SELECT ... ORDER BY` read them back in order. The `?` is a **placeholder**: the value `"Mango"` is sent separately from the SQL.

- Look in the `notes-board` folder. A new file `try.sqlite` appeared. **This one file is the whole database.**
- Run `node try.js` **again**. You now see **four** rows and `Rows in the table: 4`. The old rows were **kept** in the file, and the script added two more. This is what a database does. (`CREATE TABLE IF NOT EXISTS` did not make a second table.)

Try a filter and a delete

- Add these lines at the **end** of `try.js`:

```
const mango = db.prepare("SELECT id, name FROM fruits WHERE name = ?").all("Mango");
console.log("Mangoes:", mango);

const removed = db.prepare("DELETE FROM fruits WHERE name = ?").run("Banana");
console.log("Rows deleted:", removed.changes);
```

- Save and run `node try.js`.

Check: `Mangoes:` shows only the Mango rows (this is `WHERE`). `Rows deleted:` shows how many Banana rows were removed (this is `DELETE ... WHERE`, and `changes` is the number of rows removed).

Clean up

- Delete both practice files: `try.js` and `try.sqlite`. In VS Code, right-click each file and choose **Delete**.

Check: Only your Notes Board files remain in the folder (and `node_modules` if it is there). Your instructor will explain the question below. Think about it:

If the user types `'); DELETE FROM notes; --` as a note title, why is our `?` placeholder code safe?

Part 2: Write `data.js` with SQLite (20 minutes)

We replace the **inside** of `data.js`. The three function names stay the same, and we add one new function, `init()`.

1. Open `data.js`.
2. **Select everything (Ctrl + A)** and delete it. The file is now empty.
3. Type the code below, **one section at a time**. Your instructor will explain each section before you type it.

Section 1: Open the database

```
// data.js (Day 4)
//
// The notes now live in a real database: SQLite.
// SQLite keeps everything in one file (notes.sqlite), so the notes
// stay even when the server stops.
//
// The three functions have the same names as on Day 3,
// so server.js only needs a small change.

const { DatabaseSync } = require("node:sqlite");

const databaseFile = process.env.DATABASE_FILE || "notes.sqlite";
const db = new DatabaseSync(databaseFile);
```

What it does: it loads SQLite and opens the file `notes.sqlite`. If the file does not exist, SQLite creates it. (`process.env.DATABASE_FILE` lets us choose another file name in `.env`. We do not set it, so the default is used.)

Section 2: `init()`, make the table and add starter notes

Type this **below** Section 1:

```

// Create the table (if it does not exist yet) and add 3 starter notes
// when the table is empty. This also makes a fresh server useful at once.
async function init() {
  db.exec(`
    CREATE TABLE IF NOT EXISTS notes (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      text TEXT NOT NULL,
      created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
    )
  `);

  const row = db.prepare("SELECT COUNT(*) AS total FROM notes").get();
  if (row.total === 0) {
    const insert = db.prepare("INSERT INTO notes (title, text) VALUES (?, ?)");
    insert.run("Welcome", "This note is stored in a SQLite database.");
    insert.run("Study plan", "Learn HTML on Monday, CSS on Tuesday, and JavaScript on Wednesday.");
    insert.run("Idea", "Add your own notes. They stay after you restart the server.");
  }
}

```

The SQL is inside **backticks** (the ``` key, usually above the Tab key). Backticks let one piece of text go over many lines.

What it does: it makes the `notes` table, counts the rows, and only if the count is 0 it adds 3 starter notes.

Section 3: The three functions and the export

Type this **below** Section 2:

```
// Give back all notes, newest first.
async function listNotes() {
  return db.prepare("SELECT id, title, text FROM notes ORDER BY id DESC").all();
}

// Save a new note and give it back (with its new id).
async function addNote(title, text) {
  const result = db
    .prepare("INSERT INTO notes (title, text) VALUES (?, ?)")
    .run(title, text);
  return { id: Number(result.lastInsertRowid), title: title, text: text };
}

// Remove a note. Gives back true if it existed, false if it did not.
async function deleteNote(id) {
  const result = db.prepare("DELETE FROM notes WHERE id = ?").run(id);
  return result.changes > 0;
}

module.exports = { init, listNotes, addNote, deleteNote };
```

What it does: `listNotes` runs a `SELECT`. `addNote` runs an `INSERT` with two `?` placeholders and returns the new note. `deleteNote` runs a `DELETE` and says `true` if at least one row was removed. The last line shares **four** functions. Before, it shared three.

4. Save the file (**Ctrl + S**).

Check:

- Your `data.js` has no red underline in VS Code.
- You can find these in the file: `CREATE TABLE`, `INSERT`, `SELECT`, `DELETE`, and three `?` marks that are used as placeholders for values.
- You did **not** join any user text into a SQL string.

Part 3: Change `server.js` (7 minutes)

`server.js` needs three small changes.

Change 1. At the top, change the first comment line to:

```
// server.js (Day 4)
```

Change 2. Find this line:

```
const { listNotes, addNote, deleteNote } = require("./data");
```

and replace it with:

```
const { init, listNotes, addNote, deleteNote } = require("./data");
```

Change 3. Go to the **end** of the file. Find this block:

```
app.listen(PORT, function (error) {
  if (error) {
    console.error("Could not start the server: " + error.message);
    console.error("Is another server already using port " + PORT + "?");
    process.exit(1);
  }
  console.log("Notes Board is running. Open http://localhost:" + PORT);
});
```

Replace it with:

```
// Prepare the database first, then start listening.
init().then(function () {
  app.listen(PORT, function (error) {
    if (error) {
      console.error("Could not start the server: " + error.message);
      console.error("Is another server already using port " + PORT + "?");
      process.exit(1);
    }
    console.log("Notes Board is running. Open http://localhost:" + PORT);
  });
});
```

Save the file.

Also update `.env.example` (this file is safe to share). Add these lines at the **end** of `.env.example` :

```
# Optional. Where the SQLite file is kept (default: notes.sqlite).
# DATABASE_FILE=notes.sqlite

# Only for the PostgreSQL track (Track B):
# DATABASE_URL=postgresql://workshop:your-password@localhost:5432/notesboard
```

(Lines that start with `#` are comments. They do nothing yet.) Do **not** put your real password in `.env.example`.

Check: Your `server.js` has the word `init` in two places: the `require` line, and `init().then(`.

Part 4: Test it, restart it, and the notes stay (10 minutes)

1. In the terminal, make sure you are in the `notes-board` folder. Run:

```
node server.js
```

2. The terminal prints `Notes Board is running`. Open `http://localhost:3000`.
3. Look in the `notes-board` folder. A new file `notes.sqlite` appeared. This is your database.
4. Open `http://localhost:3000`. You see **three starter notes**: Welcome, Study plan and Idea.
5. Add a new note. Use the password from your `.env` file. The note appears at the top.
6. Delete the note named **Idea**. It disappears.
7. Try to add a note with a **wrong password**. You see an error message, and the note is not added.

The big test: restart

8. Go to the terminal and press **Ctrl + C**. The server stops.
9. Run `node server.js` again.
10. Refresh the browser with **F5**.

Check: The note you added is **still there**, and the note you deleted is **still gone**. On Day 3 this was impossible, because the notes were in memory. Now they are in the file `notes.sqlite`. This is the goal of Part A of the day.

Think: Why did the 3 starter notes not come back after the restart? Because `init()` adds them only when the table is empty.

If the notes reset to the three starter notes after a restart: you probably started the server from a different folder, so it made a second `notes.sqlite`. See the Stuck Sheet.

Stop the server with **Ctrl + C** before you go on.

Part 5: Git, make the first commit (15 minutes)

Now we save the project with Git. All Git commands are typed in the terminal, inside the `notes-board` folder.

1. Start tracking the folder:

```
git init
```

Git says `Initialized empty Git repository`. It may also print a few lines that start with `hint:`. This is normal.

2. Name the main branch:

```
git branch -M main
```

(This command prints nothing when it works.)

3. Tell Git who you are. Use your own name and the email of your GitHub account. The quotation marks are needed.

```
git config user.name "Your Name"
git config user.email "you@example.com"
```

4. See what Git sees:

```
git status
```

Check: The list of untracked files (shown in red or after `??`) includes `.gitignore`, `data.js`, `server.js`, `package.json`, `package-lock.json`, `public` and `.env.example`. It does **not** include `.env`, `node_modules` or `notes.sqlite`. These three are ignored, because `.gitignore` lists them. **This is what we want.**

If `.env`, `node_modules` or `notes.sqlite` **do** appear in the list: **stop**. Do not run `git add`. See the Stuck Sheet.

5. Prove that Git knows about the ignored files:

```
git status --ignored
```

Check: At the end there is a section named `Ignored files`. It lists `.env`, `node_modules/` and `notes.sqlite`.

6. Stage everything, then commit:

```
git add .
git commit -m "First commit: Notes Board with SQLite"
```

Check: Git prints a line with `First commit: Notes Board with SQLite` and a list of created files. If it says `Author identity unknown`, repeat step 3.

7. Look at the history:

```
git log --oneline
```

Check: You see one commit with your message.

8. Run `git status` once more.

Check: It says `nothing to commit, working tree clean`.

Part 6: Create a GitHub repository and push (15 minutes)

Signing in: When you push for the first time, Git for Windows may open a **browser window** and ask you to sign in to GitHub. **This has not been tested on the lab computers.** If it does not work, raise your hand. Your instructor will help. See the Stuck Sheet.

6a. Create an empty repository on GitHub

1. Open `https://github.com` in the browser and **sign in**.
2. Click the **New** button (or the **+** menu at the top right, then **New repository**).
3. **Repository name:** `notes-board`
4. Choose **Public**, unless your instructor tells you something different.
5. **Do not** tick **Add a README file**. **Do not** add a `.gitignore` or a licence. We want an **empty** repository.
6. Click **Create repository**.
7. GitHub shows a page with some commands, and an address that ends in `.git`. It looks like this:

```
https://github.com/YOUR-USERNAME/notes-board.git
```

Copy that address. (`YOUR-USERNAME` is your own GitHub user name.)

6b. Connect and push

In the terminal, type the next command. Replace the address with **your** address:

```
git remote add origin https://github.com/YOUR-USERNAME/notes-board.git
```

Check that the address is right:

```
git remote -v
```

Check: You see `origin` two times, with **your** user name in the address. If the address is wrong, see the Stuck Sheet (wrong remote URL).

Make sure the branch is named `main` :

```
git branch -M main
```

Now push:

```
git push -u origin main
```

If a browser window opens, sign in to GitHub and allow the connection. Then go back to the terminal.

Check: The terminal shows lines about writing objects, and at the end something like `branch 'main' set up to track 'origin/main'`. There is no line that starts with `fatal:` or `error:`.

Part 7: Verify that `.env` is NOT on GitHub (5 minutes)

This is the most important check of the day.

1. In the browser, refresh your repository page: `https://github.com/YOUR-USERNAME/notes-board`.
2. Look at the list of files and folders.

Check, you must SEE these:

```
public (folder)
.gitignore
.env.example
data.js
package.json
package-lock.json
server.js
```

Check, you must NOT see these:

```
.env
node_modules
notes.sqlite
```

3. Click `.env.example` . It shows only the fake value `change-this-password` . This is correct and safe.

If you see `.env` on GitHub: **tell your instructor now**. Do not hide it and do not wait. You will need to change your password. See the Stuck Sheet.

Show your instructor or a neighbour that `.env` is not on your page.

Part 8 (optional): Track B, PostgreSQL for fast finishers

This part is optional. Only do it if Parts 0 to 7 are finished and checked. Day 5 works without it.

PostgreSQL is a database that runs as a **separate server program**. The installer steps below are described **only in a general way**.

Check these screens against the real installer in your pilot test. They are not verified. The words on the real screens may be different. If you are not sure, ask your instructor.

The `data.js` for PostgreSQL was tested on PostgreSQL 16 on Linux only. It is not tested on Windows.

8a. Work in a copy

We do **not** want to change the project that is on GitHub.

1. In File Explorer, copy your whole `notes-board` folder and name the copy `notes-board-pg` .
2. Open `notes-board-pg` in the terminal (`cd ..` and then `cd notes-board-pg`).
3. **Do not run any `git` commands in this copy.**

4. Stop any server that is running (**Ctrl + C**). Both versions use port 3000.

8b. Install PostgreSQL (general steps, not verified)

1. Ask your instructor where the PostgreSQL installer for Windows is. It can also come from the official PostgreSQL website. Use only the file your instructor gives you.
2. Run the installer and keep the default choices.
3. The installer is expected to ask for a **password** for the main database user (called `postgres`). Choose a **simple password with only letters and numbers**, and write it down. Special characters can break the connection address.
4. The installer is expected to show a **port** number. Keep the default. The port used in our tests is `5432`.
5. You may be offered extra tools. A tool to run SQL (such as `pgAdmin` or the `SQL Shell` (`psql`)) is useful. You need one of them to make a database.
6. Finish the installation.

8c. Create the database (general steps, not verified)

1. Open the SQL tool you installed, and sign in with the `postgres` user and your password.
2. Run this SQL:

```
CREATE DATABASE notesboard;
```

8d. Connect the Notes Board

In the terminal, inside `notes-board-pg`:

1. Install the PostgreSQL package for Node.js:

```
npm install pg
```

2. Open `.env` in the copy and add this line at the end. Replace `YOUR-PASSWORD` with the password of the `postgres` user:

```
DATABASE_URL=postgresql://postgres:YOUR-PASSWORD@localhost:5432/notesboard
```

3. Rename the SQLite `data.js` so that you keep it. In the terminal:

```
ren data.js data-sqlite.js
```

4. Copy the PostgreSQL file into place. Open the folder `Day-4-Database-and-Git\done\notes-board\optional-postgres` (from the workshop folder your instructor gave you), copy `data.js`, and paste it into `notes-board-pg`.
5. Read the new `data.js`. Compare it with the SQLite one. Find the differences: `Pool`, `$1` and `$2` instead of `?`, and `SERIAL` instead of `AUTOINCREMENT`.
6. Run the server:

```
node server.js
```

Check: The page at `http://localhost:3000` shows three starter notes. Add a note, press **Ctrl + C**, run `node server.js` again, and refresh. Your note is still there, now stored in PostgreSQL.

Important: Do **not** copy this PostgreSQL `data.js` into your real `notes-board` folder. Day 5 uses the SQLite version.

Part 9: Quiz and homework (5 minutes)

1. Do the 5-question quiz that your instructor shares.
 2. Make sure you know your **GitHub user name** and password. **You need them on Day 5.**
 3. Read the **Homework for Day 5** at the end of the Day 4 Theory, including the hosting reading list.
-

Final checklist

My Notes Board keeps its notes after I stop and restart the server.

`data.js` uses SQLite with `?` placeholders, and has `init()`.

`git log --oneline` shows my commit.

My code is on GitHub, and `.env`, `node_modules` and `notes.sqlite` are **not** there.

I know my GitHub user name and password for Day 5.